SQL: Single-Table Queries and Aggregation

Databases and Software Architecture – Aalborg University

Claude, a large language model developed by Anthropic, was used to assist with structuring and grammar checking of this material. All generated text was critically reviewed, edited, and verified by the author.

Single-Table Queries and Aggregation

Last lecture gave us seven tables and every constraint that keeps their data honest. Today we start actually **asking questions** of that data: filtering rows, sorting and paging results, summarizing with aggregate functions, and grouping rows into categories. Everything today touches exactly **one** table at a time – combining data across tables comes later in the course.

Agenda

Six parts: a recap, `WHERE` and its operators, sorting and limiting, a break, aggregate functions, and `GROUP BY`/`HAVING`.

- Recap: what we already have to work with
- `SELECT` / `FROM` / `WHERE`: comparison, logical, and special operators, and NULL's three-valued logic
- `ORDER BY`, `LIMIT`/`OFFSET`, aliases, NULLs in sorting
- Break
- Aggregate functions: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`
- `GROUP BY` and `HAVING`, and the SQL execution order that explains why they behave the way they do

Recap and Motivation

Recap: The Library Schema

Seven tables, all built last lecture: `Author`, `Book`, `Member`, `BookGenre`, `Copy`, `BookAuthor`, `Loan`. Every query today runs against this same schema – no new tables, no `CREATE TABLE` today.

- `Book(book_id, title, isbn, publication_year)`
- `Author(author_id, first_name, last_name, birth_year)`
- `Member(member_id, first_name, last_name, email, join_date)`
- `Loan(loan_id, member_id, book_id, copy_number, loan_date, due_date, return_date)`
- Plus `BookGenre`, `Copy`, `BookAuthor` – the junction/weak-entity tables from Rule 3 and Rule 5

Recap: What We Could Already Do

The first lecture previewed `SELECT`, `WHERE`, and `ORDER BY` in passing, on `Student`/`Programme`. That is exactly where we pick back up – except now with a schema we understand deeply, and every clause explained properly.

```
SELECT * FROM Book;
```

```
SELECT first_name, last_name
FROM Student
WHERE programme_id = 1;
```

Today formalizes both of these, then goes well beyond them: pattern matching, ranges, NULL-aware conditions, sorting, paging, and summarizing data across many rows at once.

Today’s Question: How Do We Ask Real Questions of Our Data?

A `CREATE TABLE` statement is a promise about **shape**. Today is about **retrieval**: turning a question a librarian might actually ask into a precise, executable `SELECT` statement.

- “Which books came out in the 1950s?”
- “Which loans are still outstanding?”
- “How many books do we have per genre?”
- “Which genres have more than five books?”

Every one of these becomes a single SQL statement by the end of today.

SELECT, WHERE, and Filtering Rows

The Anatomy of a `SELECT` Statement

```
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

- `SELECT` – which **columns** to return (or `*` for all of them)
- `FROM` – which **table** to read from
- `WHERE` – which **rows** to keep, evaluated per row, before any result is returned

All three examples of `WHERE` operators today plug into that same third line.

Step 1: SELECT and FROM

```
SELECT title, publication_year
FROM Book;
```

No WHERE yet – every row of Book comes back, but only the two named columns.

Read aloud: “give me the title and publication year of every book.”

Step 2: Filtering with `WHERE`

```sql
SELECT title, publication_year
FROM Book
WHERE publication_year > 1950;
```

Now only rows where the condition evaluates to true survive. `WHERE` is checked **per row**, independently – never against the result set as a whole.

Comparison Operators

Six operators, all working exactly as expected on numbers, dates, and text:

Operator	Meaning
=	equal to
!= (or <>)	not equal to
<	less than
>	greater than
<=	less than or equal to
>=	greater than or equal to

```
SELECT last_name, birth_year FROM Author WHERE birth_year < 1900;
```

Logical Operators: AND and OR

Combine multiple conditions into one WHERE clause. AND requires both sides true; OR requires at least one side true.

```
-- AND: both conditions must hold
SELECT title FROM Book
WHERE publication_year < 1950 AND isbn IS NOT NULL;

-- OR: either condition is enough
SELECT title FROM Book
WHERE publication_year < 1940 OR publication_year > 1990;
```

Parentheses matter once AND and OR mix in the same query – AND binds tighter than OR, exactly like && and || in Java.

Logical Operator: NOT

Negates whatever condition follows it.

```
SELECT title FROM Book
WHERE NOT (publication_year < 1950);
```

Equivalent to `WHERE publication_year >= 1950` here – `NOT` is most useful when the condition it negates is not a simple comparison, such as `NOT (genre IN (...))` or `NOT (title LIKE '...')`.

Special Operator: LIKE

Pattern matching on text, using two wildcards: `%` matches **any number** of characters (including zero), _ matches **exactly one** character.

```
-- starts with "Tol"
SELECT first_name, last_name FROM Author WHERE last_name LIKE 'Tol%';

-- ends with "kien" -- % on both sides means "contains"
SELECT first_name, last_name FROM Author WHERE last_name LIKE '%kien%';

-- second letter is unknown, rest is "nimal Farm"
SELECT title FROM Book WHERE title LIKE '_nimal%';
```

`LIKE` is case-sensitive in PostgreSQL by default – `ILIKE` is the case-insensitive variant, worth knowing even though we will not need it in today's examples.

Special Operator: IN

Shorthand for “matches any value in this list” – avoids a long chain of ORs.

```
SELECT book_id, genre
FROM BookGenre
WHERE genre IN ('Fantasy', 'Science Fiction', 'Mystery');
```

Exactly equivalent to `WHERE genre = 'Fantasy' OR genre = 'Science Fiction' OR genre = 'Mystery'` – `IN` is purely readability, not a new capability.

Special Operator: `BETWEEN`

Shorthand for an inclusive range check on two ends.

```
SELECT title, publication_year
FROM Book
WHERE publication_year BETWEEN 1940 AND 1960;
```

Equivalent to `WHERE publication_year >= 1940 AND publication_year <= 1960` – **inclusive** on both ends, a common source of off-by-one mistakes if forgotten.

Query	Illustrative result (excerpt)
`title, publication_year`	The Fellowship of the Ring, 1954
	Nineteen Eighty-Four, 1949
	Foundation, 1951

Special Operator: `IS NULL` / `IS NOT NULL`

`NULL` means “no value recorded” – it is not zero, not an empty string, and critically, `= NULL` **never** matches it (more on that in a moment). Testing for `NULL` needs its own operator.

```sql
-- active loans: never returned
SELECT loan_id, member_id, book_id, due_date
FROM Loan
WHERE return_date IS NULL;
```

loan_id	member_id	book_id	due_date
1	1	1	2026-07-15
2	2	3	2026-07-19
4	4	8	2026-07-04

`Loan.return_date` is exactly the nullable column this schema was built with in mind – `NULL` there **means** “still on loan,” not “returned on an unknown date.”

Why `= NULL` Never Works

A genuinely common beginner mistake, worth calling out explicitly:

```
WHERE return_date = NULL       -- WRONG: always evaluates to unknown, matches
nothing
WHERE return_date IS NULL      -- RIGHT
```

`NULL` represents an **unknown** value – comparing anything to "unknown" with `=` yields "unknown," not "true," so the row is silently dropped. `IS NULL` / `IS NOT NULL` are the only correct way to test for it.

Why = NULL Never Works - Part 2

This is not just an = quirk: SQL uses **three-valued logic**. Every condition evaluates to TRUE, FALSE, or UNKNOWN – not just the two you might expect – and most logical operations that touch a NULL also come out UNKNOWN. A WHERE clause only keeps rows where the condition is TRUE; both FALSE and UNKNOWN rows are dropped, which is exactly why comparisons involving NULL so often vanish silently. The next few slides show precisely which combinations of AND, OR, and NOT stay decidable and which collapse to UNKNOWN.

Three-Valued Logic: AND

AND	TRUE	FALSE	NULL
TRUE	TRUE	FALSE	NULL
FALSE	FALSE	FALSE	FALSE
NULL	NULL	FALSE	NULL

FALSE AND anything is always FALSE – a single known-false condition is enough to decide the whole expression, NULL or not. But TRUE AND NULL and NULL AND NULL both come out NULL: with nothing false to rule it out, “unknown combined with true (or with another unknown)” stays unknown.

Three-Valued Logic: OR

OR	TRUE	FALSE	NULL
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	NULL
NULL	TRUE	NULL	NULL

Mirror image of AND: TRUE OR anything is always TRUE – one known-true condition is enough, regardless of what the other side is. FALSE OR NULL and NULL OR NULL, with nothing true to fall back on, come out NULL.

Three-Valued Logic: NOT

Operand	NOT
TRUE	FALSE
FALSE	TRUE
NULL	NULL

NOT cannot turn "unknown" into a definite answer either way, so NOT NULL (the three-valued **logical** operator, not the NOT NULL **constraint** keyword) is still NULL. Consequence for WHERE: NOT (return_date = NULL) is just as broken as return_date = NULL itself – negating UNKNOWN stays UNKNOWN, so the row is still silently dropped either way. PostgreSQL's docs give the same three tables, with more detail on how UNKNOWN interacts with other operators.

Recap

Category	Operators
Comparison	`=`, `!=`, `<`, `>`, `<=`, `>=`
Logical	`AND`, `OR`, `NOT`
Pattern matching	`LIKE` with `%` and `_`
Set membership	`IN (...)`
Range	`BETWEEN ... AND ...`
NULL testing	`IS NULL`, `IS NOT NULL`

All of these combine freely in one `WHERE` clause – next, controlling the **order** and **size** of the result.

Ordering, Limiting, and Aliases

ORDER BY: Sorting Results

WHERE decides **which** rows come back; ORDER BY decides what **order** they come back in. Ascending (ASC) is the default – DESC reverses it.

```
SELECT title, publication_year
FROM Book
ORDER BY publication_year DESC;
```

title	publication_year
Harry Potter and the Philosopher's Stone	1997
The Fellowship of the Ring	1954
Foundation	1951

ORDER BY: Multiple Columns

Ties on the first column are broken by the second, and so on – each column can pick its own direction.

```
SELECT last_name, first_name, birth_year
FROM Author
ORDER BY birth_year ASC, last_name ASC;
```

Read aloud: “sort by birth year, oldest first; whenever two authors share a birth year, sort those alphabetically by surname.”

LIMIT: Capping the Result Size

Returns only the first **N** rows of the (already sorted, already filtered) result.

```
SELECT title, publication_year
FROM Book
ORDER BY publication_year DESC
LIMIT 3;
```

`LIMIT` without `ORDER BY` is close to meaningless – “the first 3 rows” of an unordered result is whatever order PostgreSQL happens to store them in, which is not guaranteed to be stable.

OFFSET: Skipping Rows

Combines with LIMIT to page through a large result set – “give me the next page.”

```
-- page 2, 3 results per page: skip the first 3, take the next 3
SELECT title
FROM Book
ORDER BY book_id
LIMIT 3 OFFSET 3;
```

This exact pattern – LIMIT + OFFSET, both driven by page number – is how most web application result pages are implemented under the hood.

Column Aliases: `AS`

Renames a column (or an expression) in the **result**, without touching the underlying table.

```
SELECT last_name AS surname, birth_year AS born
FROM Author
WHERE birth_year < 1900;
```

surname	born
Tolkien	1892
Christie	1890

`AS` is optional in PostgreSQL (`last_name surname` also works) but writing it out is clearer, and required by some other DBMSs – worth the habit.

Table Aliases

The same idea, applied to a table name instead of a column – barely visible in a one-table query, but essential once a later lecture introduces joins between several tables.

```
SELECT b.title, b.publication_year
FROM Book AS b
WHERE b.publication_year > 1950;
```

`b.title` – “the `title` column, from the table aliased `b`.” Harmless bookkeeping today; load-bearing syntax starting next lecture.

NULLs in `ORDER BY`

Sorting has to put `NULL` somewhere – PostgreSQL's rule, by default: `NULL` sorts as if it were **larger** than every other value, in both directions.

```
SELECT title, publication_year FROM Book ORDER BY publication_year ASC;
-- ... 1997, then finally: Undated Reprint | NULL   (NULLs LAST)

SELECT title, publication_year FROM Book ORDER BY publication_year DESC;
-- Undated Reprint | NULL, then: 1997, 1954, ...      (NULLs FIRST)
```

So `ASC` puts `NULL`s at the **bottom**, `DESC` puts them at the **top** – both consistent with "`NULL` counts as the biggest value." Override explicitly with `ORDER BY publication_year ASC NULLS FIRST` if the default is not what a query needs.

Putting It Together

```sql
SELECT title, publication_year
FROM Book
WHERE publication_year IS NOT NULL
ORDER BY publication_year DESC
LIMIT 5 OFFSET 0;
```

Read top to bottom: filter out books with no recorded year, sort the rest newest-first, take the top five. `WHERE`, `ORDER BY`, and `LIMIT`/`OFFSET` combine in any one query, in this order in the **text** – next block asks whether that is also the order PostgreSQL **executes** them in.

Recap

- `ORDER BY column [ASC|DESC]`, multiple columns break ties left to right
- `LIMIT n` caps the result size; `OFFSET m` skips the first `m` rows – together, pagination
- `AS` renames a column or table in the result, purely cosmetic, but good practice
- `NULL` sorts as the largest value by default – last in `ASC`, first in `DESC`

Break

5-Minute Break

Stretch, grab water, ask now if any operator from the first half feels shaky. Next: aggregate functions – summarizing many rows into one number, and the single most common `NULL`-related gotcha in this course.

Aggregate Functions

Aggregate Functions: Overview

Everything so far returns **rows**. Aggregate functions collapse many rows into **one summary value** – a count, a total, an average.

Function	**Computes**
`COUNT(*)`	number of rows
`COUNT(column)`	number of **non-NULL** values in that column
`SUM(column)`	total of a numeric column
`AVG(column)`	average of a numeric column
`MIN(column)` / `MAX(column)`	smallest / largest value

Used alone, an aggregate function collapses the **entire table** to one row. `GROUP BY`, covered next, collapses it to one row **per group** instead.

COUNT(*): Counting Rows

Counts **every row**, regardless of what any column contains – including rows full of NULLs.

```
SELECT COUNT(*) AS total_loans
FROM Loan;
```

total_loans
142

An illustrative count – read as “how many loan records exist, full stop,” with no filtering at all.

`COUNT(column)`: Counting Non-NULL Values

Counts only the rows where **that specific column** is not `NULL`. Rows where it is `NULL` are silently skipped – no error, no warning.

```
SELECT COUNT(return_date) AS returned_loans
FROM Loan;
```

returned_loans
97

Fewer than `COUNT(*)`'s total, by exactly the number of loans still outstanding (`return_date IS NULL`).

The Classic Gotcha: `COUNT(*)` vs. `COUNT(column)`

The single most common aggregate-function mistake: assuming `COUNT(*)` and `COUNT(any_column)` always agree. They only agree when that column has **no** `NULL`s. A tiny concrete example, four loans:

loan_id	member_id	book_id	return_date
1	1	3	2026-05-14
2	2	5	NULL
3	3	2	2026-06-01
4	4	8	NULL

Two of the four loans are still outstanding – `return_date` is `NULL` on those rows.

The Gotcha, Resolved

Run both aggregates against that same four-row table:

```
SELECT COUNT(*) AS all_loans, COUNT(return_date) AS returned_loans
FROM Loan;
```

all_loans	returned_loans
4	2

`COUNT(*)` = 4 – every row counts. `COUNT(return_date)` = 2 – only the rows where that column is not `NULL`. Same table, same query shape, genuinely different answers – always ask **which** one a question actually needs. “How many loans exist” is `COUNT(*)`; “how many loans have been returned” is `COUNT(return_date)`.

SUM: Totalling a Numeric Column

Adds up a numeric column across all matching rows. `NULL` values are skipped, same as `COUNT(column)`.

```sql
-- total days borrowed, across every loan on record
SELECT SUM(due_date - loan_date) AS total_days_borrowed
FROM Loan;
```

total_days_borrowed
1988

`due_date - loan_date` – subtracting two `DATE` values in PostgreSQL yields a plain integer number of days.

AVG: Averaging a Numeric Column

The mean of a numeric column – again, NULLs are excluded from both the sum **and** the count used to compute it.

```
SELECT AVG(due_date - loan_date) AS avg_loan_days
FROM Loan;
```

avg_loan_days
14.6

An average loan period of roughly two weeks – exactly the kind of number a library would track over a whole semester’s data.

MIN and MAX: Extremes

Work on numbers, dates, and text alike (text compares alphabetically). NULLs are excluded, just like every other aggregate function here.

```
SELECT MIN(join_date) AS earliest_member, MAX(join_date) AS newest_member
FROM Member;
```

earliest_member	newest_member
2022-09-01	2026-03-15

The library's very first registered member, and its most recent – one query, both ends of the range.

Recap

Function	Key behaviour
`COUNT(*)`	Counts rows – `NULL`s included
`COUNT(column)`	Counts non-`NULL` values only
`SUM` / `AVG`	Ignore `NULL`s in the calculation
`MIN` / `MAX`	Ignore `NULL`s; work on numbers, dates, or text

Used alone, every one of these collapses the whole table to a single row. Next: doing that **per group** instead.

GROUP BY and HAVING

GROUP BY: Grouping Rows

Splits the table into buckets by a column's value, then applies an aggregate function **within each bucket** separately – one output row per distinct value.

```
SELECT genre, COUNT(*) AS num_books
FROM BookGenre
GROUP BY genre;
```

Every row with the same `genre` value collapses into one summary row for that genre.

`GROUP BY` in Practice

genre	num_books
Fantasy	38
Science Fiction	31
Dystopian	12
Mystery	9

An illustrative excerpt – one row per genre, each `COUNT(*)` computed only over that genre's rows in `BookGenre`.

GROUP BY with Multiple Aggregates

Any number of aggregate functions can appear alongside GROUP BY in the same SELECT list, each computed per group.

```
SELECT member_id, COUNT(*) AS num_loans, AVG(due_date - loan_date) AS avg_days
FROM Loan
GROUP BY member_id;
```

member_id	num_loans	avg_days
1	6	13.2
2	4	15.0
3	9	14.8

One row per member, both a count and an average computed independently within each member's group of loans.

HAVING: Filtering **Groups**

WHERE filters individual rows, **before** grouping happens. HAVING filters **groups**, **after** aggregation – and it is the only place an aggregate function is allowed to appear in a filtering condition.

```sql
SELECT genre, COUNT(*) AS num_books
FROM BookGenre
GROUP BY genre
HAVING COUNT(*) > 5;
```

Read aloud: “group books by genre, then keep only the genres with more than 5 books.”

HAVING in Practice

```sql
SELECT genre, COUNT(*) AS num_books
FROM BookGenre
GROUP BY genre
HAVING COUNT(*) > 1
ORDER BY num_books DESC;
```

genre	num_books
Fantasy	38
Science Fiction	31
Dystopian	12

Genres with only one book on record – however many there might be – simply do not appear in this result.

The SQL Execution Order

Here is the fact that explains **why** `WHERE` cannot use an aggregate but `HAVING` can: SQL is executed in a different order than it is written.

1	**2**	**3**	**4**	**5**	**6**	**7**
`FROM`	`WHERE`	`GROUP BY`	`HAVING`	`SELECT`	`ORDER BY`	`LIMIT`

`WHERE` runs **before** `GROUP BY` even exists – there is no aggregate to test yet, so referencing one there is a plain error. `HAVING` runs **after** grouping, once aggregates are already computed – exactly where testing one belongs.

The Classic Mistake: `WHERE` vs. `HAVING`

Trying to filter on an aggregate with `WHERE`:

```
SELECT genre, COUNT(*)
FROM BookGenre
WHERE COUNT(*) > 1
GROUP BY genre;
```

```
ERROR:  aggregate functions are not allowed in WHERE
```

PostgreSQL rejects this outright – at the point `WHERE` is evaluated, per the execution order above, `COUNT(*)` has not been computed yet. It simply does not exist there.

The Fix: WHERE vs. HAVING

Move the exact same condition to HAVING, and it works – because by then, grouping and aggregation have already happened:

```
SELECT genre, COUNT(*) AS num_books
FROM BookGenre
GROUP BY genre
HAVING COUNT(*) > 1;
```

Rule of thumb: filtering on a **raw column** – WHERE. Filtering on the **result of an aggregate function** – HAVING. WHERE and HAVING can also both appear in the same query, each doing its own job at its own stage.

Recap

- `GROUP BY column` – one summary row per distinct value, aggregates computed within each group
- `HAVING condition` – filters **groups**, evaluated after aggregation, may reference aggregate functions
- `WHERE condition` – filters **rows**, evaluated before grouping, may **not** reference aggregate functions
- The execution order (`FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY` → `LIMIT`) is **why**

The Full SELECT Syntax

```
SELECT column1, column2, ...    -- required
FROM table_name                 -- required, for every query in this course
WHERE condition                 -- optional
GROUP BY column1, ...           -- optional
HAVING condition                -- optional, only meaningful together with GROUP BY
ORDER BY column1, ...           -- optional
LIMIT n                         -- optional
OFFSET m                        -- optional
;
```

This is the order you **type** them in – fixed by SQL's grammar, and the order PostgreSQL's parser expects, error or not. Every query today has used some subset of this skeleton, always in this relative order.

`SELECT` Clauses: Required vs. Optional

Clause	Required?	Purpose
`SELECT`	Required	which columns to return
`FROM`	Required	which table to read from
`WHERE`	Optional	filter rows, before grouping
`GROUP BY`	Optional	collapse rows into groups
`HAVING`	Optional	filter groups, after aggregation
`ORDER BY`	Optional	sort the result
`LIMIT`	Optional	cap the number of rows returned
`OFFSET`	Optional	skip the first rows

This **typed** order is not the order PostgreSQL **evaluates** them in (the execution-order slide from a moment ago): getting the typed order right is a syntax requirement; keeping the execution order straight in your head is what explains why `WHERE` cannot reference an aggregate but `HAVING` can.

Wrap-Up

Introduction to the In-Class Exercise

You get a much larger version of the library database – 200+ rows across all seven tables – and 8–10 concrete questions to answer as SELECT statements, entirely with what today covered.

- “How many books do we have per genre?”
- “Which loans are currently overdue?”
- “Which members have joined in the last year?”
- Full question list and the seeded dataset are on the exercise sheet – let’s solve the first three together

Java Connection: Connecting to the Database

`WHERE column = value` looks different once it comes from application code instead of a hand-typed script – a preview of this course's later JDBC coverage. First, the connection itself:

```
// Same host, port, database, and credentials as this lecture's
// extras/docker-compose.yml -- if that container is running locally,
// this connects to it directly.
String url = "jdbc:postgresql://localhost:5432/dbs";
Connection connection = DriverManager.getConnection(url, "dbs", "dbs");
```

`DriverManager.getConnection` only works if the PostgreSQL JDBC driver (`org.postgresql:postgresql`) is actually on the **classpath** – without it, this throws `SQLException: No suitable driver found`, even though the code compiles fine and the database itself is running and reachable.

Java Connection: Parameterized Queries, Previewed

Once `connection` exists, here is that same `WHERE member_id = ?` from a moment ago, written properly:

```
String sql = "SELECT * FROM Member WHERE member_id = ?";
PreparedStatement stmt = connection.prepareStatement(sql);
stmt.setInt(1, memberId);
ResultSet rs = stmt.executeQuery();
```

The `?` is a **placeholder** – the actual `memberId` value is supplied separately, never concatenated directly into the SQL string. That separation is also what prevents SQL injection – a later lecture covers both properly.

Exercise

The next exercise session will be focused on:

- SQL tasks on the same distributed dataset, increasing in difficulty
- Early tasks: single conditions, sorting, simple aggregates
- Later tasks: combined `WHERE` conditions, `GROUP BY` with `HAVING`, careful `NULL` handling
- Full task list is on the exercise sheet – run every query against PostgreSQL and check the results make sense

Summary

Today: `SELECT`/`FROM`/`WHERE` with comparison, logical, and special operators (`LIKE`, `IN`, `BETWEEN`, `IS NULL`); `ORDER BY`, `LIMIT`/`OFFSET`, aliases, and NULL sort order; the five aggregate functions and the `COUNT(*)` vs. `COUNT(column)` distinction; `GROUP BY` and `HAVING`, and the execution order that separates them.